



Universidad Don Bosco, El Salvador

PROGRAMACIÓN CON ESTRUCTURA DE DATOS – PED

PROYECTO FASE II

“Documentación Final – Sistema de Gestión de Rutas de Entrega”

https://github.com/rm161936/ped_proyecto/tree/feature/avance2

Integrantes:

- Guillermo Alejandro Castillo Campos CC242330
- Fernando Alonso Cortez Rivas RM161936

Fecha: viernes 29 de mayo de 2026

INDICE

1. INTRODUCCION	3
2. EXPLICACIÓN DETALLADA DE LA LÓGICA	4
2.1 Situación Problemática Elegida	4
2.2 Estructuras de Datos Seleccionadas y su Forma de Aplicación	4
2.3 Procesos por Registrar – Diagramas UML	5
Diagrama de Casos de Uso	6
Diagrama de Flujo	7
Diagrama de Casos de Uso	8
2.4 Proceso de Servicio	8
3. DISEÑO DEL SISTEMA	9
3.1 Diseño de Proyecto	9
3.2 Mapa del Sistema o Diagrama Gráfico Modular	9
3.3 Descripción de los Módulos	11
3.4 Descripción de las Funcionalidades	12
3.5 Base de Datos	12
3.6 Diccionario de Datos	13
3.7 Seguridad del Sistema	16
4. FUENTES DE INFORMACIÓN	17

1. INTRODUCCION

El presente documento corresponde a la Fase II del Proyecto de Cátedra de la asignatura Programación con Estructura de Datos. En esta fase se documenta el diseño completo, la implementación técnica y el funcionamiento del sistema denominado Delivery Route Manager (Sistema de Gestión de Rutas de Entrega).

El sistema aborda una problemática real del sector logístico: la necesidad de las empresas de reparto de optimizar sus rutas de entrega, gestionar la atención de pedidos por nivel de urgencia y mantener un registro histórico de todas las operaciones. Para resolver esta problemática se aplicaron de manera directa las estructuras de datos estudiadas en la asignatura: grafos, colas de prioridad, listas enlazadas y pilas.

El sistema fue desarrollado utilizando el lenguaje de programación C# con interfaz gráfica en Windows Forms y SQLite como motor de base de datos. La arquitectura del proyecto sigue una separación clara de responsabilidades dividida en capas: modelos, estructuras de datos, algoritmos, persistencia y presentación, lo que facilita su mantenimiento y extensión.

2. EXPLICACIÓN DETALLADA DE LA LÓGICA

2.1 Situación Problemática Elegida

Las empresas de reparto a domicilio enfrentan diariamente el reto de organizar y optimizar sus rutas de entrega. Sin una herramienta adecuada, los repartidores recorren distancias innecesarias, los pedidos de mayor urgencia no se atienden con prioridad y no existe un registro ordenado de las operaciones completadas.

El sistema propuesto resuelve esta problemática permitiendo:

- Registrar los puntos de entrega y el almacén principal como nodos de un grafo.
- Establecer las rutas disponibles entre dichos puntos con su distancia en kilómetros.
- Calcular automáticamente la ruta más corta desde el almacén hasta el cliente mediante el algoritmo de Dijkstra.
- Gestionar los pedidos pendientes según su nivel de prioridad (Alta, Media, Baja) y fecha de registro.
- Mantener un historial persistente de todas las entregas realizadas.
- Revertir la última asignación de ruta mediante una funcionalidad de deshacer.

2.2 Estructuras de Datos Seleccionadas y su Forma de Aplicación

Grafo (Nodos y Aristas)

Constituye la estructura central del sistema. Cada punto de entrega y el almacén principal se representan como nodos (NodoPunto), mientras que las rutas disponibles entre ellos se representan como aristas (Arista) con un peso que corresponde a la distancia en kilómetros. El grafo es no dirigido, lo que permite transitar en ambas direcciones. Se implementa mediante la clase Grafo, que mantiene un diccionario de nodos indexado por Id, cada nodo con su lista de conexiones. Adicionalmente, el layout del grafo se calcula mediante un algoritmo force-directed que posiciona los nodos de forma clara y legible en el canvas.

Cola de Prioridad (Priority Queue)

Se utiliza para gestionar los pedidos pendientes de entrega. Cada pedido tiene un nivel de prioridad (Alta = 1, Media = 2, Baja = 3) y una fecha de registro. La cola garantiza que los pedidos más urgentes sean atendidos primero, y en caso de empate por prioridad, se respeta el orden de llegada (FIFO). Se implementa mediante un SortedList con clave compuesta (int Prioridad, DateTime Fecha), usando un comparador personalizado que ordena por prioridad y luego por fecha.

Lista Enlazada Simple

Se utiliza para almacenar el historial de entregas completadas. Cada nodo del historial (NodoHistorial) contiene los datos del pedido entregado, la ruta recorrida, la distancia total y la fecha de entrega. La inserción se realiza siempre al inicio de la lista, lo que garantiza que el historial muestre primero la entrega más reciente (orden cronológico inverso).

Pila (Stack)

Se utiliza para implementar la funcionalidad de deshacer la asignación de rutas. Cada vez que se asigna una ruta a un pedido mediante el algoritmo de Dijkstra, la acción (pedido + ruta calculada) se almacena en la pila. Si el operador necesita revertir la última asignación, se extrae el elemento superior y el pedido regresa a la cola de prioridad.

2.3 Procesos por Registrar – Diagramas UML

A continuación, se describen los procesos principales del sistema. Los diagramas UML correspondientes (diagrama de casos de uso, diagrama de flujo y diagrama de clases) se incluyen en los apartados gráficos del documento.

Proceso: Registro de punto de entrega

El operador ingresa el nombre del nuevo punto de entrega. El sistema crea un nodo en el grafo en memoria, lo persiste en la tabla Puntos de la base de datos y ejecuta el algoritmo de layout para reposicionar todos los nodos de forma legible.

Proceso: Registro de ruta entre dos puntos

El operador selecciona dos puntos existentes e ingresa la distancia en kilómetros. El sistema crea la arista bidireccional entre ambos nodos y la persiste en la tabla Rutas.

Proceso: Registro de pedido

El operador ingresa el nombre del cliente, selecciona el punto de entrega destino y asigna el nivel de prioridad. El pedido se encola en la ColaPedidos y se persiste en la tabla Pedidos.

Proceso: Cálculo y asignación de ruta (Dijkstra)

El sistema extrae el pedido de mayor prioridad de la cola. A continuación, ejecuta el algoritmo de Dijkstra sobre el grafo para calcular la ruta más corta desde el almacén hasta el punto de entrega. La ruta calculada se muestra al operador con los nodos intermedios y la distancia total. La acción se registra en la pila de deshacer.

Proceso: Completar entrega

Una vez entregado el pedido, el operador confirma la entrega. El registro se mueve de la tabla Pedidos al historial (lista enlazada en memoria y tabla Historial en la BD). El pedido es eliminado de la tabla Pedidos.

Proceso: Consulta de historial

El operador puede consultar el historial de entregas desde la base de datos en orden cronológico inverso. Dispone de filtros por nombre de cliente y por fecha de entrega.

Proceso: Deshacer asignación

Si el operador cometió un error al atender un pedido, puede deshacer la última asignación. El sistema extrae el tope de la pila y devuelve el pedido a la cola de prioridad.

Diagrama de Casos de Uso

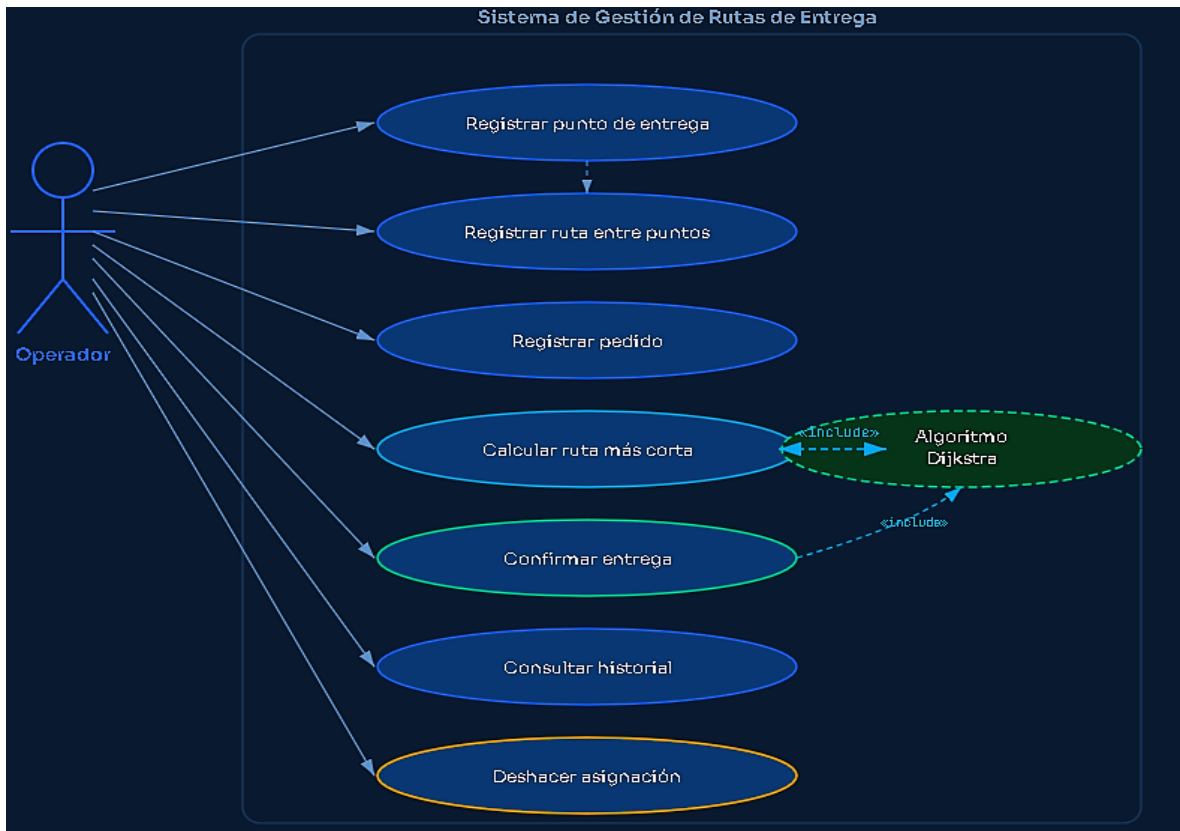


Diagrama de Flujo

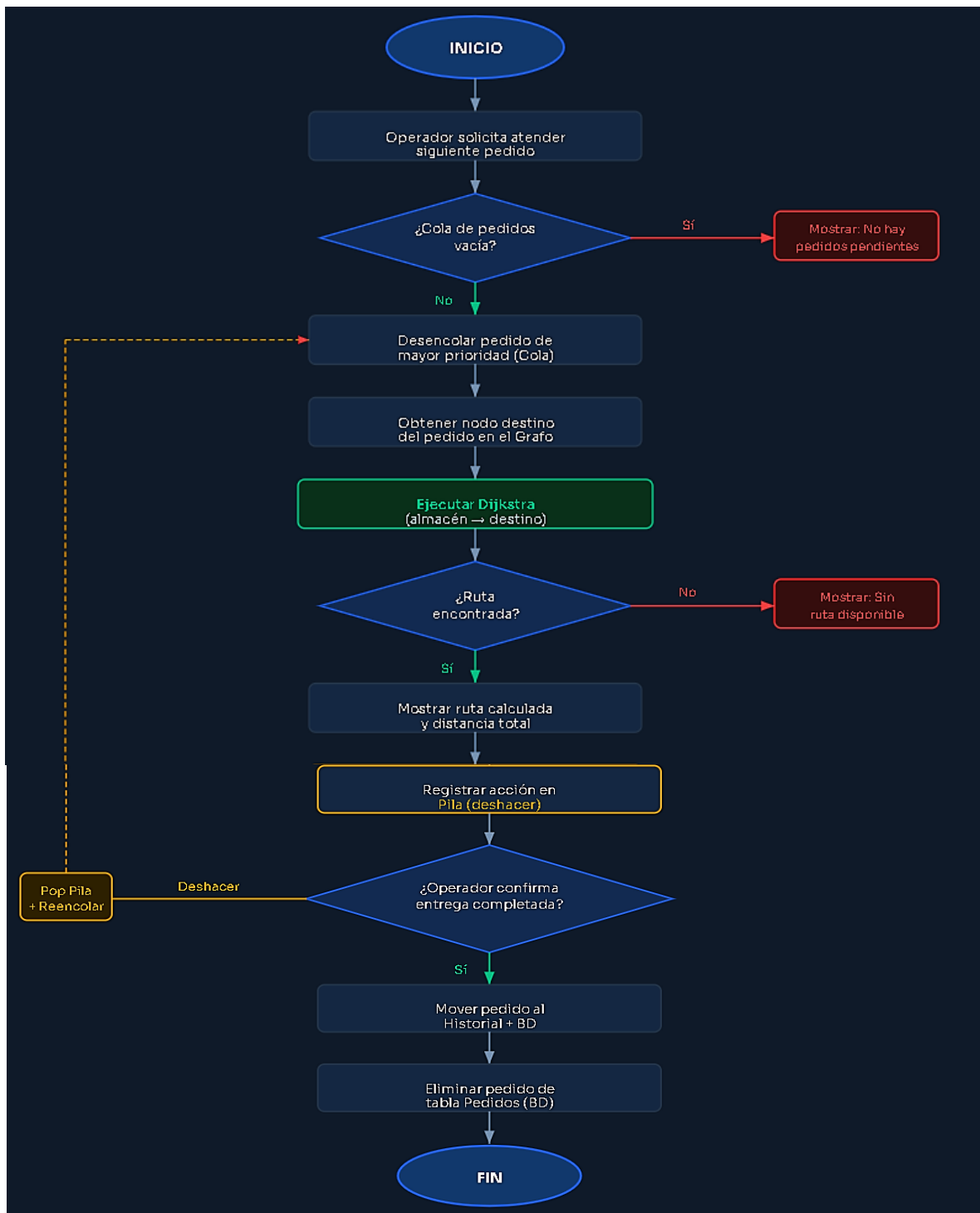
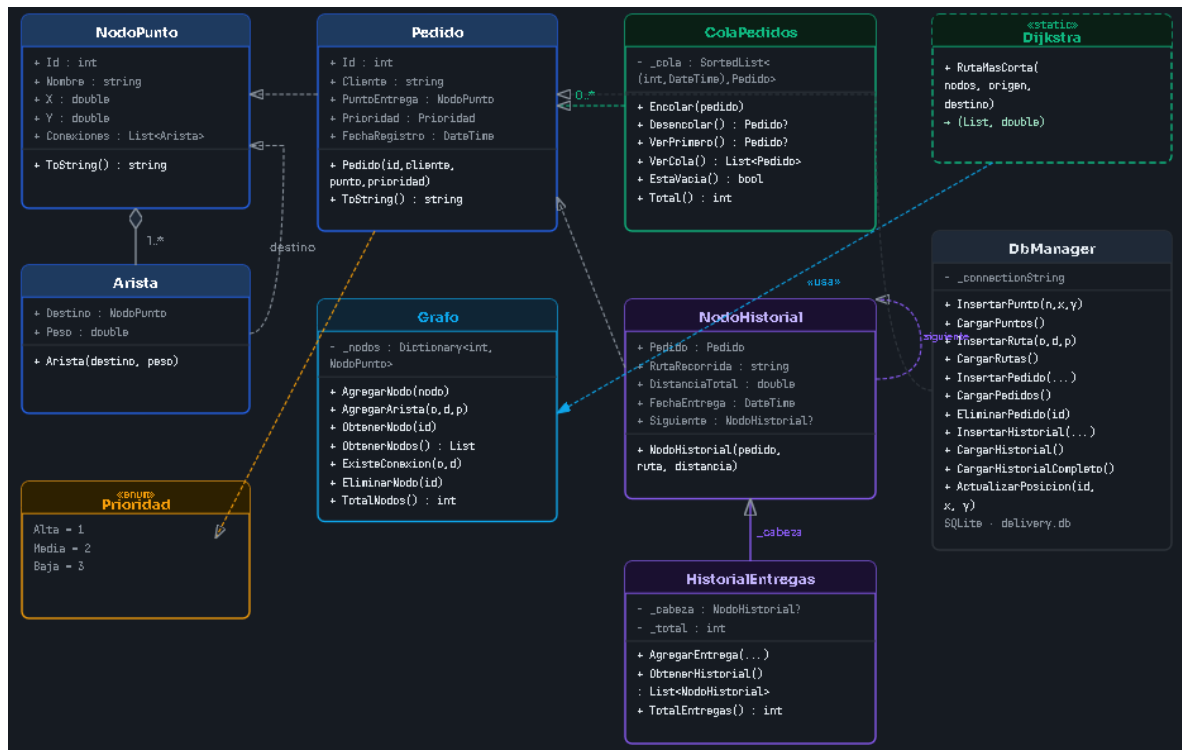


Diagrama de Casos de Uso



2.4 Proceso de Servicio

El sistema ofrece al usuario los siguientes resultados e interacciones:

- Visualización gráfica del mapa de puntos de entrega y rutas disponibles sobre un canvas interactivo con algoritmo force-directed layout.
- Cálculo automático de la ruta más corta entre cualquier par de puntos, mostrando la secuencia de nodos intermedios y la distancia total en kilómetros.
- Cola ordenada de pedidos pendientes por prioridad y fecha de registro, visible en todo momento.
- Registro persistente de pedidos que sobrevive al reinicio de la aplicación.
- Historial completo de entregas realizadas con posibilidad de filtrar por cliente o fecha.
- Función de deshacer para revertir la última asignación de ruta en caso de error del operador.

3. DISEÑO DEL SISTEMA

3.1 Diseño de Proyecto

El proyecto sigue una arquitectura en capas con separación clara de responsabilidades. Cada capa tiene una responsabilidad específica y se comunica únicamente con las capas adyacentes, lo que facilita el mantenimiento y la extensibilidad del sistema.

Capa	Carpeta	Responsabilidad
Modelos	Models/	Entidades del dominio: NodoPunto, Arista, Pedido, Prioridad.
Estructuras de Datos	DataStructures/	Implementaciones propias: Grafo, ColaPedidos, HistorialEntregas, NodoHistorial.
Algoritmos	Algorithms/	Lógica de cálculo: Dijkstra para ruta más corta.
Persistencia	Database/	Acceso a datos con SQLite: DbManager con todos los métodos CRUD.
Presentación	Forms/	Interfaz gráfica: FormPrincipal, FormGrafo, FormPedidos, FormHistorial.

El estado central de la aplicación (Grafo, ColaPedidos, HistorialEntregas y la pila de deshacer) se instancia en FormPrincipal y se pasa por constructor a cada formulario hijo. Esto garantiza un estado compartido y consistente durante toda la sesión.

3.2 Mapa del Sistema o Diagrama Gráfico Modular

El sistema está compuesto por un formulario MDI principal (FormPrincipal) que actúa como contenedor y punto de coordinación de tres módulos funcionales: Gestión de Rutas y Puntos, Gestión de Pedidos e Historial de Entregas. Cada módulo se abre como formulario MDI hijo y existe como máximo en una sola instancia simultánea.



Operador

CAPA DE PRESENTACIÓN

Forms/

FormGrafo

Gestión de Rutas y Puntos
Canvas interactivo (PictureBox)
Force-directed layout
Resaltado ruta óptima (azul)
Clic sobre nodos → info
Agrega puntos y rutas a BD

Módulo: Rutas y Puntos

FormPrincipal

MDI Host · Coordinador central de estado
Grafo · ColaPedidos · HistorialEntregas
Stack<Deshacer> · DbManager
AbrirFormulario<T>() — instancia única por módulo

FormPedidos

Gestión de Pedidos
Cola ordenada (DataGridView)
Dijkstra automático al atender
Confirmar entrega → Historial
Deshacer → regresa a cola
OnActivated: recarga puntos

Módulo: Pedidos

FormHistorial

Historial de Entregas Completadas
Lee siempre desde BD (CargarHistorialCompleto)
Filtro por cliente · Filtro por fecha
Detalle de ruta al seleccionar fila

Módulo: Historial

CAPA DE ESTRUCTURAS DE DATOS

DataStructures/

Grafo

Dictionary<int, NodoPunto>
No dirigido · Ponderado
Force-directed layout
EliminarNodo · ExisteConexion

ColaPedidos

SortedList<int, DateTime>, Pedido>
Prioridad 1-3 + FIFO fecha
Encolar · Desencolar · VerCola

HistorialEntregas

Lista enlazada simple
Inserción al inicio (_cabeza)
Orden cronológico inverso
AgregarEntrega · ObtenerHistorial

Stack · Deshacer

Stack<(Pedido, List<NodoPunto>)>
LIFO · Revertir asignación
Push(acción) · Pop() → reencolar

CAPA DE ALGORITMOS & MODELOS

Algorithms/ · Models/

«static» Dijkstra

RutaMasCorta(nodos, origen, destino)
→ (List<NodoPunto>, double distancia)
 $O(n^2)$ · Grafo ponderado no dirigido

«models» NodoPunto · Arista · Pedido · NodoHistorial · Prioridad (enum)

Entidades del dominio con constructor primario (C# 12)
Tipos base compartidos por todas las capas superiores
Alta=1 · Media=2 · Baja=3 · ToString() · Conexiones: List<Arista>

CAPA DE PERSISTENCIA

Database/ SQLite delivery.db

DbManager

InsertarPunto · CargarPuntos · InsertarRuta · CargarRutas · InsertarPedido · CargarPedidos · EliminarPedido · InsertarHistorial · CargarHistorialCompleto · ActualizarPosi

delivery.db

3.3 Descripción de los Módulos

Módulo	Archivo	Descripción
FormPrincipal	FormPrincipal.cs	Contenedor MDI principal. Instancia y coordina el estado compartido. Controla la navegación entre módulos evitando instancias duplicadas.
Gestión de Rutas y Puntos	FormGrafo.cs	Permite agregar puntos de entrega y rutas entre ellos. Muestra el grafo visualmente con layout automático. Permite calcular la ruta más corta con Dijkstra y resalta el resultado en el canvas.
Gestión de Pedidos	FormPedidos.cs	Permite registrar nuevos pedidos con cliente, punto y prioridad. Muestra la cola ordenada. Permite atender el siguiente pedido (Dijkstra automático), confirmar la entrega y deshacer la última asignación.
Historial de Entregas	FormHistorial.cs	Muestra todas las entregas completadas leídas directamente desde la BD. Permite filtrar por cliente o fecha. Muestra el detalle de la ruta recorrida al seleccionar una fila.

3.4 Descripción de las Funcionalidades

Funcionalidad	Módulo	Descripción
Agregar punto de entrega	FormGrafo	Crea un nodo en el grafo y lo persiste en la BD. Reposiciona el layout automáticamente.
Agregar ruta	FormGrafo	Crea una arista bidireccional con peso (km) entre dos nodos existentes.
Calcular ruta más corta	FormGrafo / FormPedidos	Ejecuta el algoritmo de Dijkstra y resalta la ruta óptima en el canvas. En FormPedidos se ejecuta automáticamente al atender el siguiente pedido.
Registrar pedido	FormPedidos	Encola un pedido con cliente, destino y prioridad. Lo persiste en la BD.
Atender siguiente pedido	FormPedidos	Desencola el pedido de mayor prioridad, calcula la ruta con Dijkstra y la muestra al operador.
Confirmar entrega	FormPedidos	Mueve el pedido al historial (BD y lista enlazada), lo elimina de la tabla de pedidos.
Deshacer asignación	FormPedidos	Extrae el tope de la pila y devuelve el pedido a la cola.
Consultar historial	FormHistorial	Lee las entregas completadas desde BD en orden cronológico inverso.
Filtrar historial	FormHistorial	Permite filtrar entregas por nombre de cliente y/o fecha de entrega.
Persistencia entre sesiones	DbManager	Todos los puntos, rutas, pedidos pendientes e historial se cargan al iniciar la aplicación desde SQLite.

3.5 Base de Datos

Descripción General

El sistema utiliza SQLite como motor de base de datos, a través de la librería Microsoft.Data.Sqlite. La base de datos se almacena en un archivo local denominado delivery.db ubicado en el directorio de ejecución de la aplicación. La gestión completa del acceso a datos

está centralizada en la clase DbManager, que se encarga de inicializar las tablas, migrar columnas faltantes en tablas existentes y exponer métodos CRUD para cada entidad.

La base de datos está compuesta por cuatro tablas: Puntos, Rutas, Pedidos e Historial. Las tablas Rutas y Pedidos tienen llaves foráneas hacia Puntos. La tabla Historial almacena una copia desnormalizada de los datos del pedido en el momento de la entrega, garantizando la integridad del registro histórico incluso si el punto de entrega es eliminado posteriormente del grafo.

3.6 Diccionario de Datos

A continuación se describe cada campo de las cuatro tablas del sistema.

Tabla: Puntos

Campo	Tipo de Dato	Restricción	Descripción
Id	INTEGER	PK, AUTOINCREMENT, NOT NULL	Identificador único del punto de entrega.
Nombre	TEXT	NOT NULL	Nombre descriptivo del punto (ej. Almacén, Casa Federico).
X	REAL	NOT NULL	Coordenada horizontal del nodo en el canvas del grafo (píxeles).
Y	REAL	NOT NULL	Coordenada vertical del nodo en el canvas del grafo (píxeles).

Tabla: Rutas

Campo	Tipo de Dato	Restricción	Descripción
Id	INTEGER	PK, AUTOINCREMENT, NOT NULL	Identificador único de la ruta.
IdOrigen	INTEGER	FK → Puntos(Id), NOT NULL	Referencia al punto de origen de la ruta.
IdDestino	INTEGER	FK → Puntos(Id), NOT NULL	Referencia al punto de destino de la ruta.
Peso	REAL	NOT NULL	Distancia en kilómetros entre los dos puntos. Representa el peso de la arista en el grafo.

Tabla: Pedidos

Campo	Tipo de Dato	Restricción	Descripción
Id	INTEGER	PK, AUTOINCREMENT, NOT NULL	Identificador único del pedido.
Cliente	TEXT	NOT NULL	Nombre del cliente que realizó el pedido.
IdPunto	INTEGER	FK → Puntos(Id), NOT NULL	Punto de entrega destino del pedido.
Prioridad	INTEGER	NOT NULL	Nivel de prioridad del pedido: 1 = Alta, 2 = Media, 3 = Baja.

Campo	Tipo de Dato	Restricción	Descripción
FechaRegistro	TEXT	NOT NULL	Fecha y hora de registro del pedido en formato 'yyyy-MM-dd HH:mm:ss'.

Tabla: Historial

Campo	Tipo de Dato	Restricción	Descripción
Id	INTEGER	PK, AUTOINCREMENT, NOT NULL	Identificador único del registro de entrega.
IdPedido	INTEGER	NOT NULL	Identificador original del pedido entregado.
Cliente	TEXT	NOT NULL	Nombre del cliente. Copia desnormalizada para preservar el histórico.
IdPunto	INTEGER	NOT NULL	Identificador del punto de entrega destino.
Prioridad	INTEGER	NOT NULL	Prioridad del pedido al momento de la entrega: 1 = Alta, 2 = Media, 3 = Baja.
FechaRegistroPedido	TEXT	NOT NULL	Fecha y hora en que fue registrado originalmente el pedido.
RutaRecorrida	TEXT	NOT NULL	Secuencia de puntos de la ruta asignada (ej. Almacén → Bodega 1 → Casa Federico).

Campo	Tipo de Dato	Restricción	Descripción
DistanciaTotal	REAL	NOT NULL	Distancia total recorrida en kilómetros según la ruta calculada.
FechaEntrega	TEXT	NOT NULL	Fecha y hora en que el operador confirmó la entrega.

3.7 Seguridad del Sistema

En su versión actual, el sistema no implementa un módulo de autenticación de usuarios, dado que está concebido para uso local en una estación de trabajo de un único operador. Sin embargo, se aplican las siguientes medidas de seguridad a nivel de datos:

- Uso de parámetros en todas las consultas SQL (@param) para prevenir inyección SQL.
- La base de datos SQLite es un archivo local, por lo que el acceso físico al equipo es el control de acceso primario.
- Las validaciones de entrada se realizan en la capa de presentación antes de cualquier operación sobre el grafo o la base de datos.
- El mecanismo de migración (ALTER TABLE) protege la integridad de datos existentes al agregar nuevas columnas sin perder registros previos.

4. FUENTES DE INFORMACIÓN

Las siguientes fuentes fueron consultadas durante el diseño e implementación del sistema, citadas conforme a la norma APA 6.^a edición.

Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). Introduction to algorithms (3.a ed.). MIT Press.

Goodrich, M. T., Tamassia, R., & Goldwasser, M. H. (2014). Data structures and algorithms in Java (6.a ed.). Wiley.

Schildt, H. (2017). C# 7.0: The complete reference. McGraw-Hill Education.

Microsoft. (2023). C# documentation. Microsoft Learn. <https://learn.microsoft.com/en-us/dotnet/csharp/>

Microsoft. (2023). System.Collections.Generic namespace. Microsoft Learn. <https://learn.microsoft.com/en-us/dotnet/api/system.collections.generic>

Microsoft. (2024). Microsoft.Data.Sqlite overview. Microsoft Learn. <https://learn.microsoft.com/en-us/dotnet/standard/data/sqlite/>

SQLite Consortium. (2023). SQLite documentation. <https://www.sqlite.org/docs.html>

Dijkstra, E. W. (1959). A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1), 269–271. <https://doi.org/10.1007/BF01386390>

Fruchterman, T. M. J., & Reingold, E. M. (1991). Graph drawing by force-directed placement. *Software: Practice and Experience*, 21(11), 1129–1164. <https://doi.org/10.1002/spe.4380211102>